
Mid-Review Report: Eliminating Clipping Bias in DP-SGD

Priyanshu Agarwal
Department of Data Science and AI
IIT Madras
NS26Z015

Sudiksha Singh
Department of Data Science and AI
IIT Madras
NS26Z019

1 Introduction and Literature Survey

Deep learning models can sometimes memorize parts of their training data, especially when the data contains some sensitive information. This makes the model vulnerable to the privacy attacks such as data extraction and membership inference attack. To address this issue, Martin Abadi et al. (2016) proposed Differentially private Stochastic Gradient Descent (DP-SGD), which provides strict theoretical privacy guarantee during training phase [1].

1.1 Gradient Clipping and Clipping Bias

At its core, DP-SGD works by limiting how much influence any single training data point can have on the model. This is done by clipping the l_2 norm of the individual gradient to a fixed threshold and then adding Gaussian noise to the result [1]. For a given gradient g_t and threshold C , the operation can be expressed as:

$$\tilde{g}_t = g_t \cdot \min \left(1, \frac{C}{\|g_t\|_2} \right)$$

Though the clipping operation guarantees strict privacy bound, it is also a source of severe bias in optimization trajectory. For standard DP-SGD to converge to a local minimum, it is required that the gradient estimator must remain unbiased means the expected value of estimator must equal to the true full batch gradient: $\mathbf{E}[\tilde{g}_t] = \nabla \mathcal{L}(\theta)$.

The clipping operation is not a linear transformation, as a result of that the expectation of the clipped gradient is not equal to the clipped expectation of the true gradient. When the magnitude of the gradient exceeds threshold C , the extra directional magnitude gets discarded. Consequently, $\mathbf{E}[\tilde{g}_t] \neq \nabla \mathcal{L}(\theta)$. This proves that clipping operation introduces a persistent and systematic bias into the training loop, called as clipping bias, which slows down the convergence speed and also reduces the model utility.

1.2 Error Feedback via Dice-SGD

To resolve this problem of clipping bias without compromising the differential privacy guarantees, Zhang et al. (2024) proposed an error feedback algorithm named as Dice-SGD [6]. The Algorithm solves the problem by storing the residual error, the portion which was cut off, in a memory buffer rather than discarding it permanently.

Algorithm 2 of Zhang et al. (2024) formally defines that at iteration step t , the algorithm computes an update direction v^t by combining the clipped per-sample gradients with the clipped version of the accumulated error e^t from the previous iteration [6]:

$$v^t = \frac{1}{B} \sum_{i \in \mathcal{B}^t} \text{clip}(\nabla f(x^t; \xi_i), C_1) + \text{clip}(e^t, C_2)$$

The Model parameters x are then updated using both the direction and the added privacy Gaussian Noise w^t [6]:

$$x^{t+1} = x^t - \eta^t(v^t + w^t)$$

More importantly, instead of throwing away the unclipped gradient magnitude, the algorithm calculated the exact difference between the true raw gradients and the updated one v^t which was applied. This difference now becomes the new error e^{t+1} , which is accumulated and saved in the memory buffer for the next iteration [6]:

$$e^{t+1} = e^t + \frac{1}{B} \sum_{i \in \mathcal{B}^t} \nabla f(x^t; \xi_i) - v^t$$

In this way by delaying the application of gradient magnitude instead of permanently throwing it away, Dice-SGD algorithm maintains the expectation balance, eliminating the clipping bias and at the same time maintaining the strict (ϵ, δ) -differential privacy bounds required for the security [6].

2 Desired Outcome

The primary objective of this empirical project is to develop and evaluate the first Open Source implementation of the Dice-SGD algorithm. To achieve this goal, we will have two deliverables: Backend Software Engineering and Empirical Benchmarking.

2.1 Engineering Outcome: Custom Opacus Integration

Currently there is no publicly available implementation of the Dice-SGD algorithm. So, the primary goal of the engineering outcome is to convert the theoretical error-feedback mechanism of Dice-SGD to a practical fully working optimizer in PyTorch.

The standard Meta’s Opacus architecture intercepts the gradients via `DPOptimizer` class and `PrivacyEngine.make_private()` hook [5]. Now for implementing our Dice-SGD algorithm we need to subclass the core `DPOptimizer` to override the `step()` and `zero_grad()` methods. The major technical challenge in this is the instantiation of the internal state memory buffer and managing it, especially the accumulated tensor for holding the clipped gradient residuals (e^t), ensuring that the memory pointers of the standard PyTorch parameter updates don’t get corrupted.

Managing hardware constraints of this backend integration is also very critical. Dice-SGD requires approximately three times the memory space of a standard DP-SGD in order to store the unclipped gradients and the accumulated error feedback across batches, as mentioned by Zhang et al. (2024) [6]. Now to deal with this overhead of memory space without triggering Out Of Memory (OOM) exceptions during the calculation of computationally heavy per-sample gradient, our deployment environment strictly utilize NVIDIA T4 Tensor Core GPUs integrated with custom batch-memory tracking logic.

2.2 Empirical Benchmarking Outcome

After the successful deployment of the custom optimizer, the empirical objective is to conduct a rigorous benchmark analysis of Dice-SGD against the standard baseline SGD. The experiments will be conducted on two standard vision datasets, namely, MNIST and CIFAR-10. Initial testing will use MNIST dataset which allows for rapid iteration of the optimizer state. Then, we will do the benchmarking on CIFAR-10 dataset which will require deep Convolutional neural networks (CNNs). Extracting exact per-sample gradient requires much computational overhead as the convolutional layers rely heavily on shared network parameters [1, 5]. Successful training of CIFAR-10 will prove the model robustness for the modern architectures.

One thing to note is transitioning to CIFAR-10 brings strict architectural constraint. Standard CNN heavily utilizes Batch normalization to accelerate training, but as we know batch normalization computes statistics across entire batch which violates the sample level isolation required for privacy. So, to ensure that our custom Dice-SGD implementation remains mathematically robust within the Opacus framework, we will be required to refactor the CNN architectures by replacing all BatchNorm layers with GroupNorm or InstanceNorm. This will guarantee that the per sample gradient extracted for the error feedback memory buffer (e^t) remains independent.

For the evaluation of the robustness, we have structured a systematic grid search. Standard DP-SGD heavily degrades in the performance when the clipping norm (C) is misspecified. For proving that the DP-SGD is robust to suboptimal hyperparameters, we will train independent models across multi-dimensional grid outlined in Table 1.

Table 1: Proposed Hyperparameter Grid Search for Phase 2 Benchmarking

Hyperparameter	Proposed Search Values
Clipping Norm Threshold (C)	0.1, 0.5, 1.0, 5.0, 10.0
Target Privacy Budget (ϵ)	0.5, 2.0, 4.0, 8.0
Noise Multiplier (σ)	Dynamically calculated based on ϵ
Learning Rate (η)	0.01, 0.05, 0.1
Batch Size ($ B $)	64, 128, 256
Momentum	Fixed at 0.5 (to isolate error-feedback effects)

3 Evidence of Preliminary Progress

A baseline DP-SGD model was successfully implemented and trained prior to the mid-review, to validate our computational pipeline and get a formal control group for future benchmarking.

3.1 Experimental Setup and Hyperparameters

The preliminary baseline model was evaluated on the Standard MNIST dataset which consists of 60,000 training data points and 10,000 testing samples. We trained a Convolutional Neural Network (CNN) and utilised the Cross Entropy loss function, which provides stable gradient descent suitable for multi-class digit classification [3]. The model was wrapped using Meta Opacus PrivacyEngine module to enforce privacy during optimization [5]. The Complete PyTorch implementation, data loaders and training logs for this baseline are documented in our project repository¹.

To ensure that our model remains stable while converging quickly under noisy conditions, we chose our optimization hyperparameters based on standard best practices. We used Stochastic Gradient Descent (SGD) with a reduced momentum of 0.5 to keep noisy updates from compounding [1]. We set the learning rate to 0.05 to account for the extra variance from privacy noise [5]. We also chose a fixed batch size of 64 to balance reliability in gradient estimation with computational speed [4]. Finally, we limited the training to 5 epochs to keep the cumulative privacy loss within strict bounds.

3.2 Baseline Gradient Norm Analysis and Results

For Differential Privacy configuration, we enforced the maximum gradient clipping norm of $C = 1.0$. This threshold is widely adapted to balance out the limitation of sensitivity against the preservation of useful gradient trajectory [1, 6]. We applied a Gaussian noise multiplier of $\sigma = 1.0$ to ensure that the individual data points get masked sufficiently without destroying the model utility. Finally, the probability of target failure is set to $\delta = 10^{-5}$, which strictly adheres to the standard practise of keeping δ smaller than the inverse of the dataset size [2].

¹Code and colab Notebooks available at: [https://github.com/sudiksha-22f3001123/](https://github.com/sudiksha-22f3001123/Eliminating_Clipping_Bias_in_DP-SGD)
Eliminating_Clipping_Bias_in_DP-SGD

A critical observation during this baseline run was the behavior of unclipped per-sample gradient norm. We observed that the median gradient norm was effectively bounded by the setting $C = 1.0$ during the initial epochs, and allowing the noise multiplier ($\sigma = 1.0$) to remain small enough to preserve the utility. But, due to standard DP-SGD discards the magnitude of the gradient above 1.0 the model’s accuracy dropped significantly.

As illustrated in Figure 1, the baseline model of DP-SGD implementation successfully converged, achieving a final test accuracy of approximately 91.0%. The Opacus privacy accountant verified that this accuracy was achieved with a final privacy budget of $\epsilon \approx 0.35$.

The calculation of this privacy budget was not bounded by standard composition theorems, which often yield loose estimations. Instead, the Opacus engine utilizes the Rényi Differential Privacy (RDP) accountant which tracks the higher moments of the privacy loss random variable at each step. Over the 5 epochs the RDP moments get accumulated and then converted back to standard (ϵ, δ) -guarantees at the end of training, the framework provides a strict mathematical bound of $\epsilon \approx 0.35$.

In phase 2, after implementing the Dice-SGD error feedback memory mechanism, we expect to capture the discarded gradient magnitude which caused the drop in model accuracy, eventually pushing the the final convergence closer to the non private baseline accuracy without requiring a larger ϵ budget.

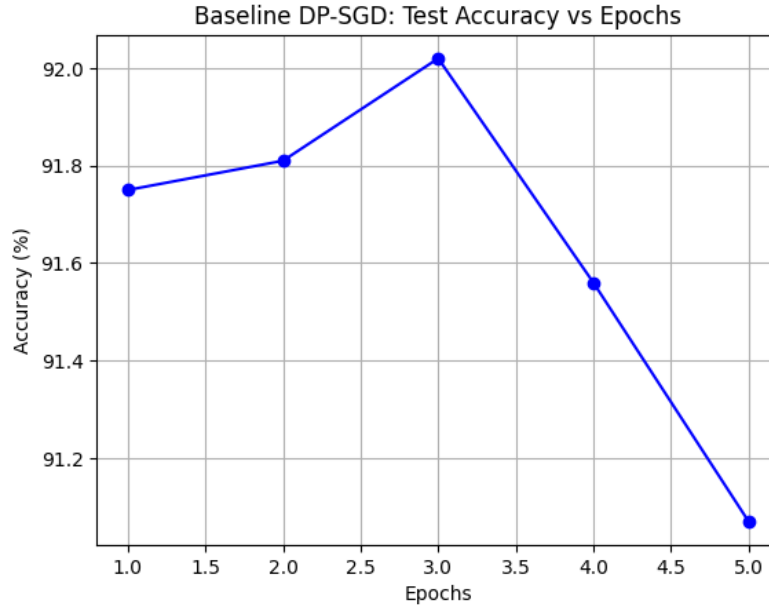


Figure 1: Baseline DP-SGD Test Accuracy vs. Epochs on MNIST dataset, demonstrating successful convergence under the strict privacy budget.

It is important to note the micro-oscillations visible in the test accuracy curve, particularly between epoch 3 and epoch 5. The standard SGD gives the smooth, monotonically increasing trajectory whereas the DP-SGD curve is volatile due to the continuous injection of Gaussian Noise ($\sigma = 1.0$) at every gradient update step. This empirical behavior confirms that the optimizer trajectory is successfully disrupted by noise multiplier, to mask individual sample contributions, functioning exactly as intended.

References

- [1] Martín Abadi, Andy Chu, Ian Goodfellow, H Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. Deep learning with differential privacy. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, pages 308–318, 2016.
- [2] Cynthia Dwork, Aaron Roth, et al. The algorithmic foundations of differential privacy. *Foundations and Trends® in Theoretical Computer Science*, 9(3–4):211–407, 2014.
- [3] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep learning*. MIT press, 2016.
- [4] Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. On large-batch training for deep learning: Generalization gap and sharp minima. *arXiv preprint arXiv:1609.04836*, 2016.
- [5] Ashkan Yousefpour, Igor Shilov, Alexandre Sablayrolles, Davide Testuggine, Karthik Prasad, Mani Malek, John Nguyen, Sayan Ghosh, Akash Bharadwaj, Jessica Zhao, et al. Opacus: User-friendly differential privacy library in pytorch. In *NeurIPS 2021 Workshop Privacy in Machine Learning*, 2021.
- [6] Xinwei Zhang, Mingyi Hong, Zhiqi Bu, and Zhiwei Steven Wu. Differentially private sgd without clipping bias: An error-feedback approach. In *The Twelfth International Conference on Learning Representations*, 2024.